Adaptive Multi-Agent NPCs: A Memory-Driven Framework for Emergent Character Behavior in Games

Shivansh Srivastava
Department of Computer Science and Engineering
Thapar University
Patiala, India
ssrivastav3_be24@thapar.edu

Aditya Gupta
Department of Computer Science and Engineering
Thapar University
Patiala, India
agupta25_be24@thapar.edu

Varish Kapoor
Department of Computer Science and Engineering
Thapar University
Patiala, India
vkapur_be24@thapar.edu

Ishpreet Singh
Department of Computer Science and Engineering
Thapar University
Patiala, India
isingh4_be24@thapar.edu

***Abstract*—Non-player characters (NPCs) in most games rely on static, hand-scripted dialogue trees that cannot adapt, remember, or grow across interactions. This work proposes a multi-agent framework in which large language model (LLM)-driven NPCs sustain persistent memory, self-reflection, and evolving personality traits through entity-level memory representations. Such agents can hold coherent, continuously evolving conversations instead of repeating fixed scripts. The architecture is anchored by an LLM gateway that assigns per-character virtual keys for fine-grained token observability and cost control. A hybrid memory router unifies summary buffers, episodic logs, and entity-based retrieval-augmented generation (RAG) for context-aware recall. A reviewer agent continuously audits dialogue for repetition, while a guardrail-bound god entity retains limited authority to influence the simulated world. The framework is validated through a working demonstration set in a medieval tavern populated by five to seven autonomous NPCs.**

***Index Terms*—multi-agent systems, non-player characters, large language models, episodic memory, retrieval-augmented generation, game AI**

I. Introduction

Modern games boast advanced graphics and physics, yet NPC behavior remains bound to static state machines and scripted dialogue that lack memory and adaptation. While LLMs enable dynamic, reasoning agents, scaling them introduces technical hurdles in token cost, dialogue loops, character consistency, and safety guardrails. This proposal outlines an architecture that treats NPCs as independent LLM agents managed by a lightweight gateway-delivering reactive game environments while generating multi-agent conversational data for scientific and reasoning research.

This proposal outlines a system that treats every NPC as an independent LLM agent with its own memory, personality, and API budget, coordinated through a lightweight gateway and observability layer. Beyond entertainment, the same architecture doubles as a controlled environment for generating multi-agent conversational data, which is useful for behavioral-science research and for training or evaluating reasoning models.

II. Multi-Agent Framework and Implementation

The system (Fig. 1) is organized into four cooperating layers: the character-agent layer, the memory layer, the gateway/observability layer, and an oversight layer consisting of a reviewer agent and a guardrail-bound god entity.

A. Character Agents

Each NPC is an independent agent seeded with a small set of manually authored traits (personality, role, speech style). During play, an agent maintains a rolling summary buffer of the current conversation, an episodic memory of past interactions, and a self-reflection step that periodically re-evaluates and updates its own trait file based on recent experience, allowing characters to genuinely develop rather than stay fixed. A per-character session log, stored as a plain text file, is appended and rolled over on a daily/session basis, giving each NPC a durable diary that later conversations can draw upon.

B. Hybrid Memory and Retrieval

A memory router mediates every read/write between an agent and three memory tiers: the short-term summary buffer, an episodic store of past events, and a RAG vector index built over each character's session logs. When an agent needs context beyond its buffer, the router retrieves the most relevant prior episodes, giving the illusion of long-term recall without sending an ever-growing transcript to the LLM.

C. Gateway, Virtual Keys, and Conversation Control

All inference calls pass through an LLM gateway configured with a free/open LLM API and a virtual key per character,

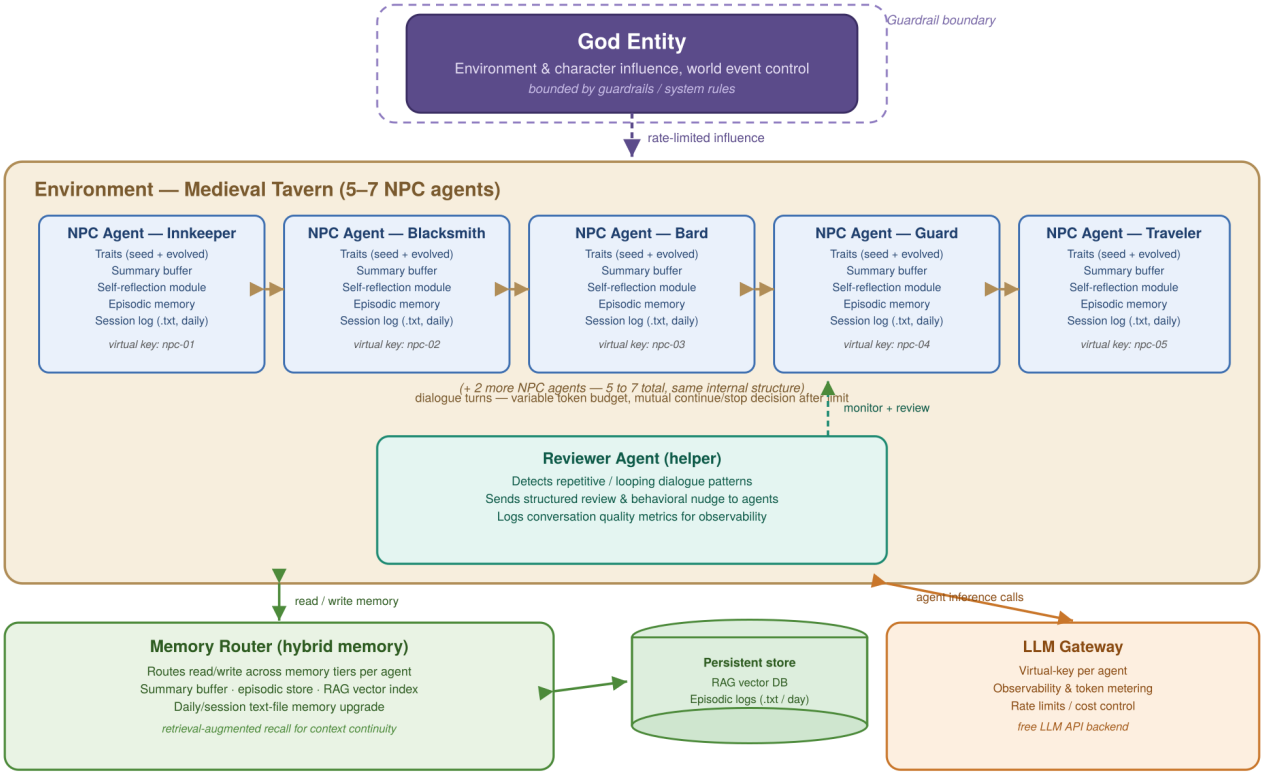

Fig. 1. Multi-agent NPC architecture with memory routing, gateway observability, and behavioral oversight.

Fig. 1. Multi-agent NPC architecture with memory routing, gateway observability, and behavioral oversight.

enabling per-agent observability, token metering, and rate limiting. Conversations between two NPCs run under a variable token budget; once the limit is reached, both participating agents independently decide whether to continue or end the exchange, avoiding artificially truncated or infinitely running dialogue.

D. Reviewer Agent and God Entity

A separate reviewer (helper) agent monitors ongoing conversations for repetition or looping and sends a structured review back to the offending agent, nudging it toward different phrasing or topics. A god entity can influence characters and the environment (e.g., triggering events), but its power is deliberately nerfed through guardrails and system rules so it cannot override character autonomy or safety constraints.

E. Demo Environment

The final demo instantiates this architecture as a medieval tavern containing five to seven NPCs (innkeeper, blacksmith, bard, guard, traveler, and others), each with a distinct seed personality. The first milestone is functional and text-based; visualization is treated as a later-stage enhancement once the reasoning, memory, and control loops are validated.

III. System Architecture

While Section II defines the conceptual multi-agent behavior model, Fig. 2 shows how that model is realized as a running software system, a self-contained virtual sandbox split across a simulation frontend and an agent backend.

A. Frontend: Unity (C#) Simulation

The tavern scene, character prefabs, and animation state machines are implemented in Unity using C#. A `CharacterController` script drives per-NPC movement and NavMesh placement, a `DialogueUI` component renders conversation turns as chat bubbles, and a `NetworkClient` script communicates with the backend over REST for request/response calls and WebSocket for streaming agent responses. The frontend holds no LLM or memory logic itself; it is a thin visualization and input layer over the backend simulation.

B. Backend: FastAPI Routing Layer

An ASGI server built with FastAPI and Uvicorn exposes a small set of asynchronous endpoints. This layer performs authentication, request logging, and CORS handling, then routes each call to the Agent Orchestration layer, decoupling the Unity client from the internal agent implementation.

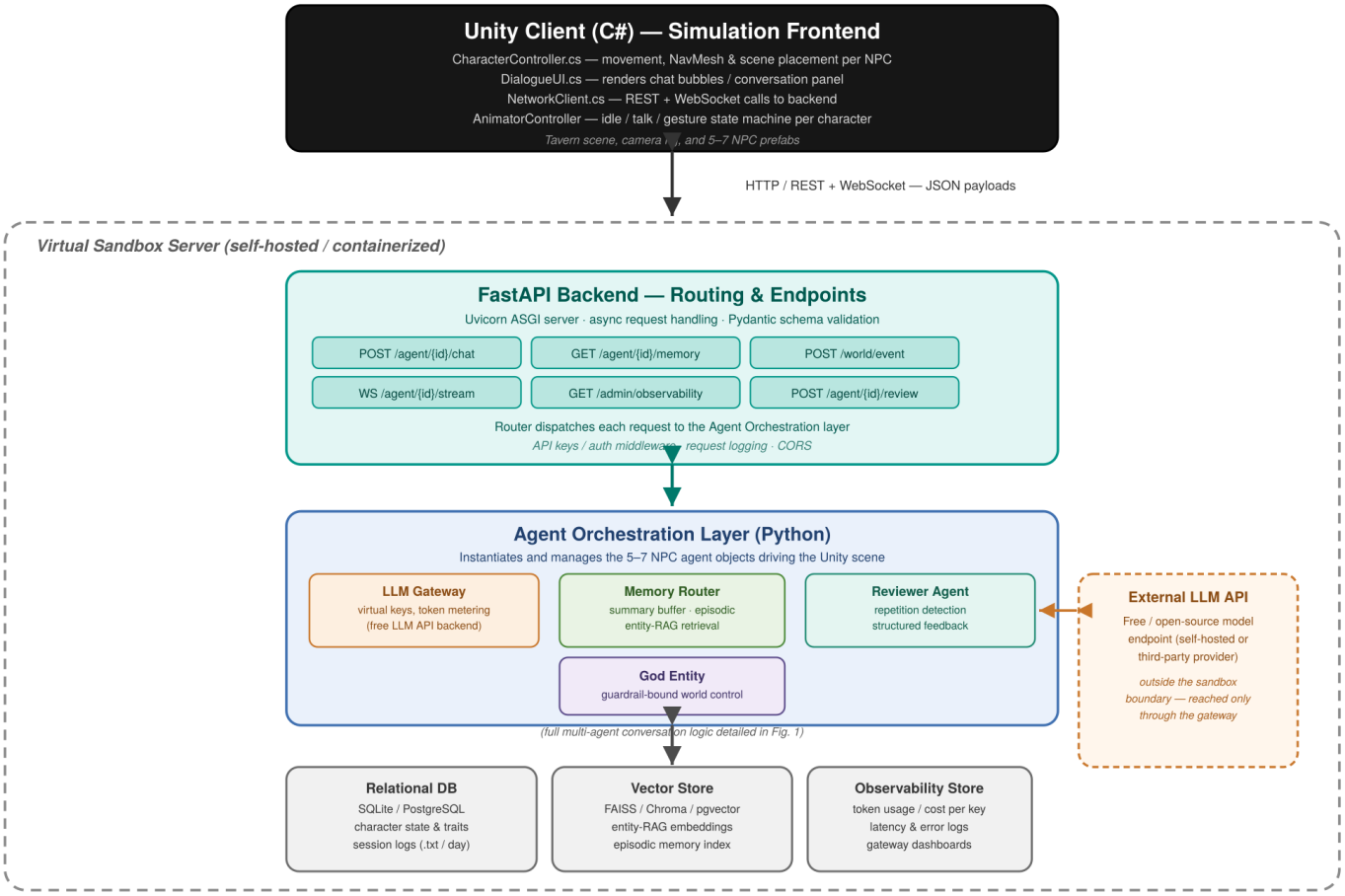

Fig. 2. Software implementation architecture: Unity (C#) simulation frontend, FastAPI routing layer, and sandboxed agent backend.

C. Agent Orchestration and Virtual Sandbox

The FastAPI router dispatches into a Python orchestration layer that instantiates and manages the NPC agent objects described in Section II, the LLM gateway (virtual keys and token metering), the memory router, the reviewer agent, and the guardrail-bound god entity. This layer, together with the routing layer and the storage services below it, runs inside a self-hosted, containerized *virtual sandbox*, isolating the simulation from the outside network except for calls the gateway explicitly makes to the external LLM API.

D. Storage and External Services

Three storage services back the orchestration layer: a relational database (SQLite/PostgreSQL) for character state, traits, and rolling session logs; a vector store (FAISS/Chroma/pgvector) holding entity-RAG embeddings for episodic retrieval; and an observability store recording token usage, cost per virtual key, and latency for each agent. The LLM itself is reached only through the gateway, keeping the sandbox boundary the single point of egress to any external API.

IV. Future Scope

- Add a visual/animated front end (2D or 3D) once the core reasoning loop is stable.
- Extend the hybrid memory with emotion and relationship tracking between characters.
- Use the framework to generate multi-agent behavioral datasets for experimental psychology and to train/evaluate reasoning models on social interaction data.
- Run human-evaluation studies comparing scripted NPCs against memory-driven agents for believability and engagement.

V. Conclusion

This project reframes NPCs as observable, memory-equipped LLM agents rather than static scripts, combining a token-aware gateway, hybrid RAG memory, and lightweight behavioral oversight (reviewer agent and guardrailed god entity) into one coherent architecture. The medieval-tavern demo will validate the approach at a small scale while leaving a clear path toward richer visualization and broader research use in behavioral data generation.

Acknowledgment

The author thanks the course instructors for guidance on scope and evaluation criteria for this proposal.